Kafka Assignment

Applied, end-to-end assessment on Confluent Cloud

1. Overview and Scenario

You are a data engineer at **VoltRoute**, an operator of a nationwide network of electric-vehicle (EV) charging stations. Every time a driver plugs in, a charging session is created; when they unplug, the session is closed with the energy delivered. The business wants a real-time streaming pipeline that ingests these events, enriches them with station reference data, and produces live operational metrics (energy consumed, active sessions, and session cost) per city and per station.

In this assignment you will design the topics, generate mock data with your own producers, register and evolve Avro schemas, run a scaled consumer group, and build the stream-processing logic in ksqlDB using streams, tables, windowed aggregations, and windowed joins. The goal is to demonstrate that you can **apply** Kafka concepts end to end, not just recall them.

What this assignment tests

- Topic design: partitioning, keying, cleanup policy, and co-partitioning for joins.
- Producer development with Avro serialization and Schema Registry.
- Backward-compatible schema evolution across producer and consumer versions.
- Consumer groups: partition assignment, scaling, rebalancing, and offset management.
- ksqlDB: streams vs tables, stream-table enrichment joins, windowed aggregations, and windowed stream-stream joins.

2. Environment Setup (Confluent Cloud only)

This assignment must run entirely on **Confluent Cloud**. Do **not** install or run a local Kafka broker, ZooKeeper, or a local Schema Registry. Your producer, consumer, and any Kafka Streams code run locally (or in any environment you like) and connect to Confluent Cloud over the network using the bootstrap endpoint and API keys.

Provision these three managed resources

- A **Kafka cluster** (a Basic cluster is sufficient). Note its bootstrap server and create a cluster API key and secret.
- **Schema Registry** (Stream Governance) for the environment. Note the Schema Registry URL and create an SR API key and secret.
- A **ksqlDB cluster**. Choose the smallest size available for this workload.

Cost note: Confluent Cloud offers free trial credits on sign-up (check the current amount shown at signup). A running ksqlDB cluster consumes credits continuously, so provision it only while working and DELETE the ksqlDB cluster (and any unused topics/clusters) when you finish each session to avoid exhausting your credits.

Sample client configuration (fill in your own values)

```
# client.properties  (used by your producer/consumer clients)
bootstrap.servers={{BOOTSTRAP_SERVERS}}
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule \
  required username='{{CLUSTER_API_KEY}}' password='{{CLUSTER_API_SECRET}}';

# Schema Registry
schema.registry.url={{SR_URL}}
basic.auth.credentials.source=USER_INFO
basic.auth.user.info={{SR_API_KEY}}:{{SR_API_SECRET}}
```

You may use any client language (Python with confluent-kafka is recommended, or Java). On a fully-managed cluster the replication factor is fixed by Confluent Cloud, so you configure partition count, keying, and cleanup policy - not replication factor.

3. Datasets, Sample Events, and Schemas (provided)

You will generate all data yourself with your producers, but you do **not** need to invent the structure - the record shapes, sample events, and Avro schemas are given below. Use exactly these field names so your ksqlDB statements line up. All three datasets are keyed as described in Part A.

3.1 charging_stations (reference / dimension data)

One record per physical station. This is slowly-changing reference data that you will model as a ksqlDB TABLE. Key = station_id.

Sample event (JSON view of the Avro value):

```
{
  "station_id": "ST-DEL-014",
  "station_name": "Connaught Place Hub",
  "city": "Delhi",
  "operator": "VoltRoute",
  "latitude": 28.6304,
  "longitude": 77.2177,
  "max_power_kw": 150.0,
  "num_connectors": 6,
  "tariff_per_kwh": 18.5,
  "status": "ACTIVE"
}
```

Avro schema (subject: charging_stations-value):

```
{
  "type": "record",
  "name": "ChargingStation",
  "namespace": "com.voltroute.reference",
  "fields": [
    {"name": "station_id",     "type": "string"},
    {"name": "station_name",   "type": "string"},
    {"name": "city",           "type": "string"},
    {"name": "operator",       "type": "string"},
    {"name": "latitude",       "type": "double"},
    {"name": "longitude",      "type": "double"},
    {"name": "max_power_kw",   "type": "double"},
    {"name": "num_connectors", "type": "int"},
    {"name": "tariff_per_kwh", "type": "double"},
    {"name": "status",         "type": "string"}
  ]
}
```

Generate 12-20 stations spread across 4-5 cities (e.g. Delhi, Mumbai, Bengaluru, Hyderabad, Pune). max_power_kw in {50, 150, 350}; tariff_per_kwh roughly 12-22; status mostly ACTIVE with a couple in MAINTENANCE. Produce each station keyed by station_id (upsert semantics).

3.2 session_start (event stream)

Emitted when a vehicle begins charging. Key = session_id. This schema is deliberately the **v1** version; you will evolve it in Part C.

Sample event:

```
{
  "session_id": "SESS-000187",
  "station_id": "ST-DEL-014",
  "connector_id": 3,
  "vehicle_id": "EV-45211",
  "user_id": "USR-9032",
  "start_time": 1751692800000,
  "start_soc_pct": 24
}
```

Avro schema v1 (subject: session_start-value):

```
{
  "type": "record",
  "name": "SessionStart",
  "namespace": "com.voltroute.events",
  "fields": [
    {"name": "session_id",   "type": "string"},
    {"name": "station_id",   "type": "string"},
    {"name": "connector_id", "type": "int"},
    {"name": "vehicle_id",   "type": "string"},
    {"name": "user_id",      "type": "string"},
    {"name": "start_time",
       "type": {"type": "long", "logicalType": "timestamp-millis"}},
    {"name": "start_soc_pct","type": "int"}
  ]
}
```

start_time is epoch milliseconds. start_soc_pct (state of charge) is 5-60. connector_id is 1..num_connectors of that station.

3.3 session_stop (event stream)

Emitted when the vehicle unplugs, seconds to minutes after the matching start. Key = session_id (same as its start, so the two co-partition for the windowed join).

Sample event:

```
{
  "session_id": "SESS-000187",
  "station_id": "ST-DEL-014",
  "stop_time": 1751696400000,
  "end_soc_pct": 82,
  "energy_kwh": 41.6,
  "stop_reason": "COMPLETED"
}
```

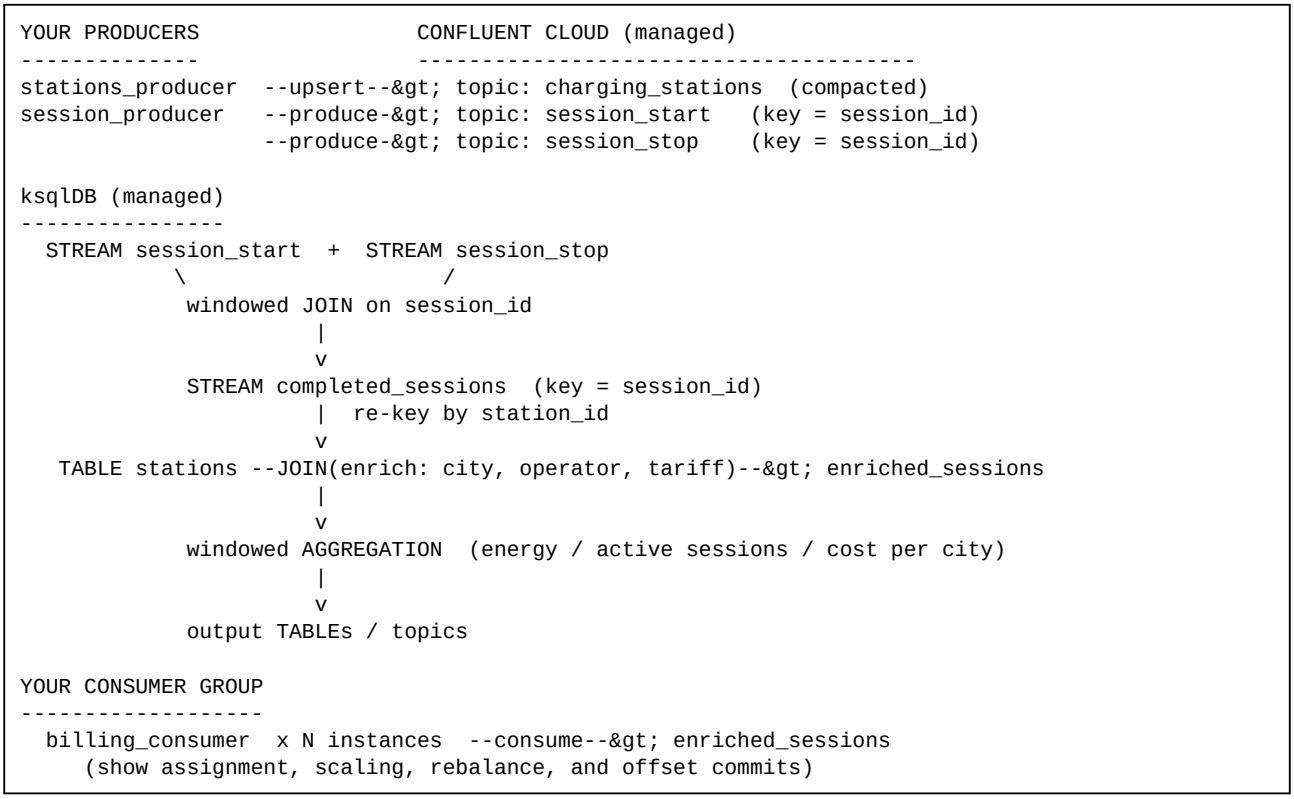
Avro schema (subject: session_stop-value):

```
{
  "type": "record",
  "name": "SessionStop",
  "namespace": "com.voltroute.events",
  "fields": [
    {"name": "session_id",  "type": "string"},
    {"name": "station_id",  "type": "string"},
    {"name": "stop_time",
       "type": {"type": "long", "logicalType": "timestamp-millis"}},
    {"name": "end_soc_pct", "type": "int"},
    {"name": "energy_kwh",  "type": "double"},
    {"name": "stop_reason", "type": "string"}
  ]
}
```

stop_time is after start_time (e.g. 10-90 minutes later). end_soc_pct is greater than start_soc_pct. energy_kwh roughly 5-60. stop_reason in {COMPLETED, USER_STOPPED, FAULT}. Leave about 5-10% of starts WITHOUT a matching stop so you can observe unmatched records and window expiry.

4. Reference Data Flow

The pipeline you will build is shown below. This is a reference - your submitted architecture diagram may refine it, but it must show topics, keys, ksqlDB objects, and the consumer group.



Part A - Topic Design (10 pts)

Create the three topics on your Confluent Cloud cluster and document your design choices.

- Choose a **partition count** for the event topics and justify it based on target parallelism and future consumer scaling. Use the **same partition count** for session_start and session_stop so they co-partition on session_id.
- Choose the **key** for each topic: charging_stations keyed by station_id, session_start and session_stop keyed by session_id. Explain why keying matters for ordering and for joins.
- Set **cleanup.policy=compact** for charging_stations and explain why a reference/lookup topic should be compacted rather than deleted; use the default (delete) for the event topics.
- In your README, explain what **replication factor** and **min.insync.replicas** you would choose for durability on a self-managed cluster and why - even though Confluent Cloud manages these for you.

Part B - Producers and Mock Data (20 pts)

Write producer scripts that generate the mock data using **Avro serialization** registered against Schema Registry (use the exact schemas from Section 3).

- **Stations producer:** produce the 12-20 station records to charging_stations, keyed by station_id. Because the topic is compacted, re-running it should upsert, not duplicate.
- **Session producer:** continuously generate sessions. For each session, emit a session_start (keyed by session_id) referencing a real station_id, then after a simulated charging duration emit a matching session_stop with the same session_id, consistent station_id, higher end_soc_pct, and a realistic energy_kwh.
- Generate at least a few hundred sessions total so your windows and aggregations have meaningful data, and intentionally leave about 5-10% of starts with no matching stop.
- Confirm in the Confluent Cloud console that the subjects were registered and that messages are Avro-encoded.

Part C - Schema Registry and Evolution (15 pts)

Demonstrate a real backward-compatible schema evolution on the session_start value schema.

- Ensure the subject's compatibility mode is **BACKWARD** (the default). State in your README what BACKWARD compatibility permits and who upgrades first (consumers or producers).
- Evolve session_start to **v2** by adding the field shown below - a new field **with a default**, which is a backward-compatible change.
- Produce some events with v2 while your existing v1 consumer keeps running. Show that the v1 consumer still works, and that a v2-aware consumer reads the new field (older records surface the default value).
- Capture evidence: the registered schema versions, and consumer output before and after the change.

Field to add for v2:

```
{"name": "payment_method", "type": "string", "default": "UNKNOWN"}
// valid values you may emit: "APP_WALLET", "CREDIT_CARD", "RFID_CARD"
```

Bonus insight to include: what would happen if you instead added a field WITHOUT a default, or removed a field - and why the registry would accept or reject it under BACKWARD.

Part D - Consumer Group and Rebalancing (15 pts)

Write a consumer that reads session events (or the enriched output from Part E) and demonstrates consumer-group mechanics.

- Run the consumer as a group (same group.id) with **multiple instances** and show which partitions each instance owns.
- Scale the group up and down (add/remove an instance) and capture the **rebalance**: log the partitions revoked and assigned.
- Use **manual offset commits** (commit after processing) and explain how this gives at-least-once delivery and why the downstream must tolerate duplicates.
- In your README, explain the roles of session.timeout.ms, heartbeat.interval.ms, and max.poll.interval.ms, and how the CooperativeSticky assignor reduces disruption during a rebalance.

Part E - ksqlDB Stream Processing (30 pts)

Build the processing logic in ksqlDB. Write the statements yourself; a minimal skeleton is shown only to fix the expected object names and formats.

E.1 Declare streams and the reference table (5 pts)

- Create a STREAM over session_start and a STREAM over session_stop (VALUE_FORMAT='AVRO', keyed by session_id).
- Create a TABLE over charging_stations (keyed by station_id) representing the latest state of each station.

```
-- skeleton only; complete the column lists / options yourself
CREATE STREAM session_start_stream WITH (
  KAFKA_TOPIC='session_start', VALUE_FORMAT='AVRO', KEY_FORMAT='KAFKA');

CREATE TABLE stations_table WITH (
  KAFKA_TOPIC='charging_stations', VALUE_FORMAT='AVRO', KEY_FORMAT='KAFKA');
```

E.2 Windowed stream-stream join: match start to stop (9 pts)

- Join session_start to session_stop on **session_id** within a time **window** (choose a sensible window, e.g. sessions complete within a few hours) to produce a **completed_sessions** stream.
- Compute **duration_minutes** from start_time and stop_time and carry energy_kwh and station_id forward.
- Explain in the README why a windowed join is required here and what happens to starts that never get a matching stop within the window.

E.3 Stream-table enrichment join (8 pts)

- Re-key completed_sessions by **station_id** and join it to stations_table to enrich each completed session with city, operator, and tariff_per_kwh.
- Compute **session_cost = energy_kwh * tariff_per_kwh**.
- In the README, address **co-partitioning**: why the stream must share the table's key and partition count for this join to work, and how you satisfied it.

E.4 Windowed aggregation (8 pts)

- From the enriched stream, compute per-**city** metrics over a **tumbling window** (e.g. hourly): total energy_kwh delivered, number of completed sessions, and total session_cost.
- Additionally compute active/completed sessions per station over a **hopping window** to show you can use a different window type.

- Note the difference between tumbling and hopping windows in your README.

Part F - Kafka Streams (Bonus, +5 pts)

Optional. Reimplement **one** of the windowed aggregations from Part E.4 as a small **Kafka Streams** application (Java) that connects to your Confluent Cloud cluster, reads the relevant topic, computes the windowed aggregate into a state store, and writes results to an output topic. In the README, briefly compare the developer experience of Kafka Streams versus ksqlDB for this task.

5. Deliverables and Submission

Submit your **entire codebase** as one of the following: a **.zip archive**, a **Google Drive link**, or a **public GitHub repository**. The submission must contain everything needed to understand and re-run your work.

Required contents

- **Executable code files:** producer(s), consumer(s), and any Kafka Streams app, each runnable with clear instructions. Do not commit real API keys or secrets - use placeholders or environment variables and document them.
- **ksqlDB scripts:** all CREATE STREAM / TABLE and query statements in one or more .sql files, in the order they should be executed.
- **Avro schema files** for all three subjects, including the v2 session_start.
- **Architecture diagram** (image or PDF) showing producers, topics (with keys and partition counts), Schema Registry, the ksqlDB objects and their joins/windows, and the consumer group.
- **README** (see required sections below).

README must document

- Setup steps to reproduce (which Confluent Cloud resources to create and how to configure clients).
- Part A topic-design decisions with justification (partitions, keys, cleanup policy, and your RF / min.insync reasoning).
- How to run each script, and in what order.
- Schema-evolution proof: the versions, the compatibility mode, and before/after consumer behaviour.
- Consumer-group evidence: partition assignment and a captured rebalance.
- Explanation of each ksqlDB object: the windowed join, the enrichment join (including co-partitioning), and the windowed aggregations.
- Sample outputs (screenshots or captured records) for the enriched stream and the aggregation results.
- Any assumptions, limitations, or shortcuts you took.

6. Evaluation Rubric (100 pts + 5 bonus)

Each part is graded on correctness and on the quality of your written justification. Working code with weak reasoning does not earn full marks; the README explanations carry real weight.

Criterion	What earns the marks	Pts
A. Topic design	Correct topics; sound partition/keying/cleanup choices; RF/ISR reasoning.	10
B. Producers + Avro	Working producers; correct Avro + Schema Registry; realistic mock data.	20
C. Schema evolution	Backward-compatible v2; v1 and v2 consumers coexist; evidence captured.	15
D. Consumer group	Multi-instance group; rebalance shown; manual commits; timeouts explained.	15
E.1 Streams / table	Correct STREAM and TABLE declarations with proper keys and formats.	5
E.2 Windowed join	Correct start-stop join on session_id within a window; duration computed.	9
E.3 Enrichment join	Re-key and join to stations; cost computed; co-partitioning explained.	8
E.4 Windowed agg.	Correct tumbling and hopping aggregations per city/station.	8
Docs and diagram	Clear README covering all required sections; accurate architecture diagram.	10
Bonus: Kafka Streams	Working Streams app reproducing one aggregation; comparison written.	+5

Constraints and tips

- Keep it lean: 12-20 stations and a few hundred sessions are plenty. Depth of understanding matters more than data volume.
- Use the exact field names provided so your ksqlDB statements align with the schemas.
- Never commit secrets. Tear down the ksqlDB cluster when not in use to preserve your Confluent Cloud credits.
- Prefer clarity over cleverness - a clean, well-explained pipeline scores higher than an over-engineered one.